

Natural Language Ticket Booking System — System Design Document

Status: Ready to Deploy **Owner:** Somasekhar Thupakula **Last updated:** 21-07-26

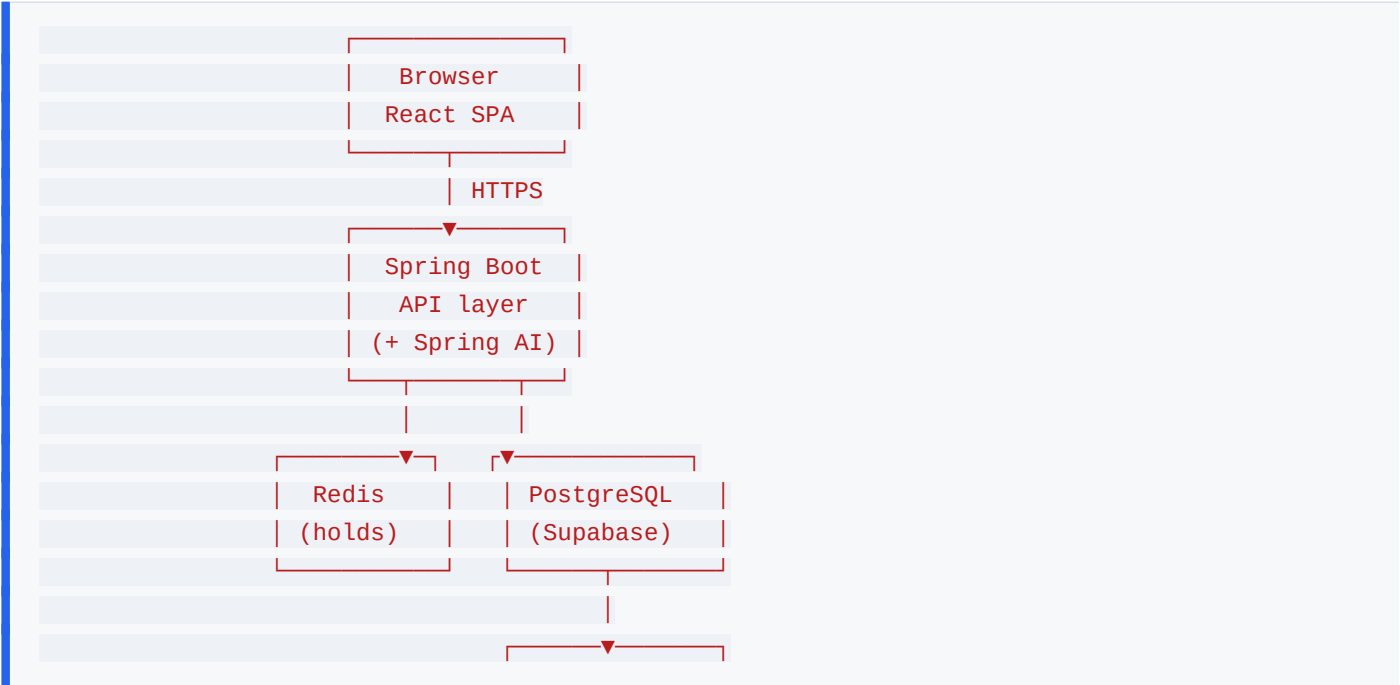
1. Executive Summary

A concurrency-safe event ticket booking platform that guarantees no seat is ever sold to more than one user, even under heavy simultaneous demand on the same seat, while offering both a conventional click-to-book UI and a natural-language AI booking assistant that shares the exact same correctness guarantees as manual booking.

Key capabilities:

- Live seat map with real-time availability sync across all connected clients
- Two-tier locking (Redis soft holds + PostgreSQL pessimistic locks) preventing double-booking
- Compensating transactions on payment failure, releasing holds automatically
- Natural-language booking via an AI agent that calls the same transactional API as the UI, never bypassing it
- Idempotent booking confirmation, safe against client retries

2. System Architecture Overview



Supabase

Realtime

Requests reach the Spring Boot API directly (fronted by the hosting platform's managed TLS/routing — see Section 8.3). The API coordinates Redis for short-lived seat holds and PostgreSQL for durable booking state, with Spring AI handling natural-language requests by invoking the same internal booking functions as the REST endpoints. Seat state changes propagate to all clients via Supabase Realtime.

3. Component Breakdown

3.1 Frontend — React / Vite SPA

Key pages:

- Event listing / search
- Seat map (live availability, hold countdown timer)
- Checkout (payment form, confirmation)
- AI chat panel (natural-language booking)

3.2 Backend — Spring Boot (Java)

Handles event/seat/booking CRUD, the two-tier hold-and-confirm locking flow, payment orchestration (mocked), and AI tool-call routing via Spring AI. Organized by feature: `event/`, `seat/`, `booking/`, `payment/`, `ai/`.

3.3 Database — PostgreSQL (Supabase-hosted)

Source of truth for events, seats, and bookings. Supabase additionally provides Change Data Capture, which powers the real-time layer without a custom WebSocket broadcaster.

3.4 Reverse Proxy

Not self-managed — TLS termination and routing are handled by the hosting platform (Railway for the API, Vercel for the frontend), consistent with the project's deployment decision to avoid self-managed infrastructure at this traffic scale. If self-hosting is adopted later, nginx would take this role, matching the pattern used in Synapse Hub.

4. Data Model

Core Entities

`events`

`id, name, venue, event_datetime, total_seats`

seats

id, event_id (FK), seat_label, status [AVAILABLE|HELD|BOOKED], version

bookings

id, user_id, event_id (FK), seat_ids, status, idempotency_key, created_at

holds (Redis, non-relational)

key: hold:{seatId} → { userId, expiresAt }

Key Relationships

- An **event** has many **seats**
- A **booking** references one **event** and one or more **seat** IDs
- A **hold** is a transient, non-relational fact about a **seat**, never the durable source of truth for whether it's sold

5. API Design

GET	/api/events	– list events
GET	/api/events/{id}/seats	– seat map for an event
POST	/api/holds	– { eventId, seatIds } → holdId, expiresAt
DELETE	/api/holds/{holdId}	– manual release
POST	/api/bookings/confirm	– { holdId, paymentDetails, idempotencyKey }
GET	/api/bookings/{id}	– booking detail
POST	/api/ai/chat	– natural-language interface, internally calls the endpoints above

6. Authentication & Authorization

6.1 Local Authentication (*Bootstrap / Temporary*)

Not implemented — this project uses Clerk directly in all environments, including local development, rather than a separate mock auth mode.

6.2 OAuth2 Authorization Code Flow (*Production Auth*)

Clerk handles sign-in and issues JWTs verified by the Spring Boot backend (signature, issuer, audience, expiry) on every request. User identity from the verified token is what's attached to bookings — never a client-supplied user ID.

6.4 Role Permissions Summary

Action	Any authenticated user
--------	------------------------

Browse events, view seat map	✓
Hold and book seats	✓
View own booking history	✓
View/modify another user's booking	✗

(This project has no admin/staff role distinction currently — flag if that's expected to change, e.g. for event organizers managing their own listings.)

7. Booking Lifecycle

AVAILABLE → HELD (Redis TTL) → BOOKED (Postgres commit)

└──────────────────┐
└──────────────────┘ → AVAILABLE (on expiry or payment failure)

A seat enters **HELD** the moment a user selects it, purely as a Redis-layer fact. It only becomes durably **BOOKED** once payment succeeds and the pessimistic-lock transaction commits in PostgreSQL. Any failure path — expired hold, failed payment, client abandonment — returns the seat to **AVAILABLE**, verified explicitly via the project's compensating-transaction tests.

8. Infrastructure & Deployment

8.1 Environment

Backend deployed to Railway, frontend to Vercel, database via Supabase-hosted PostgreSQL, Redis via Upstash — chosen over self-managed AWS infrastructure since this project isn't carrying production traffic; the CI/CD pipeline still enforces the same build-test-gate-deploy discipline a production system would need.

8.2 Container Configuration

Dockerfile for the backend; Docker Compose used for local development (Postgres + Redis + API).

8.3 Networking

TLS and routing handled by Railway/Vercel's managed infrastructure; CORS on the backend restricted to the deployed frontend origin.

8.4 Secrets Management

Clerk keys, Supabase connection string, Upstash Redis credentials, and any AI API keys stored as GitHub Actions secrets and platform environment variables — never committed to source.

8.5 Database Backup

Handled via Supabase's managed backup functionality. *(Confirm the actual retention/restore policy Supabase provides on your plan tier, and note it here explicitly rather than assuming.)*

8.6 Deployment Process

GitHub Actions: PR workflow runs build, unit tests, and concurrency stress tests as a hard gate; merge to `main` triggers deploy via Railway/Vercel's native GitHub integration, followed by a post-deploy health-check smoke test.

9. Security Design

Authentication is fully delegated to Clerk rather than hand-rolled. Authorization ensures a user can only view/act on their own bookings, checked server-side against the verified session's user ID. The correctness-critical surface of this project is concurrency safety, not access control complexity (unlike Synapse Hub) — the primary "security" property that matters most here is that the locking guarantees cannot be bypassed by any caller, including the AI agent, which is enforced by routing the AI exclusively through the same API endpoints as the UI.

10. Non-Functional Requirements

- **Correctness:** zero double-bookings under concurrent load on the same seat, verified via stress testing, not assumed
- **Real-time latency:** seat map updates visible to other clients within a low, sub-second window
- **Resilience:** payment failure must never leave a seat permanently unbookable
- **Idempotency:** duplicate confirm requests (client retries) must never result in duplicate charges or bookings

11. Known Limitations & Future Considerations

- No admin/organizer role for managing event listings — currently seeded manually
- Payment is mocked, not integrated with a real processor
- Redis hold layer is a candidate for replacement by the custom-built `mini-kv` store, gated behind the same concurrency and chaos test suite used to validate the current Redis-backed design
- Load test benchmark numbers should be published in the README once formally run, rather than left as placeholders